
Fashion Attribute Extraction & Product Description Generation

An End-to-End Fashionpedia Pipeline: Data, Model, and Serving

Project Report

Generated July 9, 2026

Abstract

This report documents an end-to-end system that turns a fashion product photograph into (a) a structured set of fine-grained garment attributes and (b) a natural-language product description. The system is built on the **Fashionpedia** dataset and consists of three parts: a **data preprocessing pipeline** that converts raw COCO-style segmentation/attribute annotations into a clean multi-label classification dataset; a **ConvNeXt-Tiny-based multi-label classifier**, partially fine-tuned with a differential learning-rate scheme to predict 101 visual attributes; and a **three-stage inference service** (detection → classification → LLM description) exposed via a FastAPI backend. We describe the dataset statistics and cleaning decisions, the model architecture and training recipe (including an ablation design over how much of the backbone to unfreeze), and report the results actually obtained by the training run in this repository, including where training was cut short and what that implies. We close with the production serving architecture, latency budget, and a candid discussion of limitations and next steps.

Contents

1	Project Overview	3
2	Dataset: Fashionpedia	3
2.1	Source and raw statistics	3
2.2	Preprocessing pipeline and design rationale	3
2.3	Label space and class imbalance	4
2.4	Processed dataset and image/category distribution	5
3	Model Architecture	6
3.1	Backbone and head	6
3.2	Partial fine-tuning and the ablation axis	7
3.3	Loss function	8
3.4	Optimizer and learning-rate schedule	8
4	Training Methodology	9
4.1	Data pipeline and augmentation	9
4.2	Metrics	9
4.3	Runs actually executed	9
5	Results	10

5.1	Training run status	10
5.2	Loss and metric curves	10
5.3	Discussion	10
6	Inference Serving Architecture	12
6.1	Stage details	12
6.2	Latency budget	13
6.3	API contract	13
7	Key Engineering Decisions — Summary	13
8	Limitations and Future Work	13
9	Conclusion	14

1 Project Overview

The goal of this project is a backend API that accepts a single product photograph and returns:

1. A list of detected **fashion attributes** (e.g. *midi length*, *floral pattern*, *A-line silhouette*), each with a confidence score and a semantic group (*supercategory*).
2. A short, natural-language **product description** synthesized from those attributes by a language model.

POST /describe $\implies$ image $\rightarrow$ {attributes[], attributes_by_group{}, description, meta}

The project spans the full ML lifecycle used in the repository:

- **Data engineering** (`fashionpedia_preprocessing.py`) — cleaning, filtering, and re-encoding the raw Fashionpedia annotation files into a supervised multi-label dataset.
- **Model training** (`train.py`) — a ConvNeXt-Tiny backbone with a custom classification head, partial fine-tuning, class-imbalance-aware loss, and a warmup+cosine learning-rate schedule.
- **Experiment analysis** (`plot_training_curves.py`) — training curve and threshold-sweep visualization from logged CSVs.
- **Serving** (`main.py`, `app/models.py`) — an async FastAPI service wrapping a three-stage inference pipeline, with a local small LLM (Qwen2.5-0.5B-Instruct) or a Claude Haiku API fallback for description generation.

Figure 1 shows the system end to end, from raw dataset to served API response.

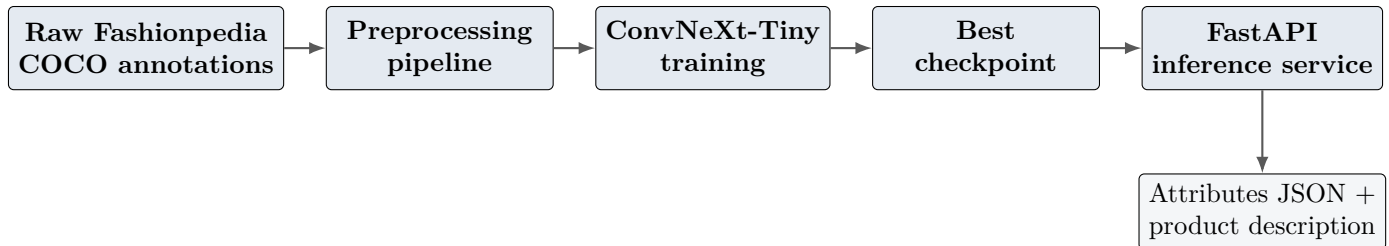


Figure 1: Top-level project flow: dataset $\rightarrow$ preprocessing $\rightarrow$ training $\rightarrow$ checkpoint $\rightarrow$ serving.

2 Dataset: Fashionpedia

2.1 Source and raw statistics

Fashionpedia annotates fashion product images at two levels: garment-part **instance segmentation** (46 categories, e.g. *sleeve*, *dress*, *neckline*) and per-instance **visual attributes** (294 fine-grained attribute labels grouped into supercategories such as *silhouette*, *length*, *textile pattern*). The project uses the `instances_attributes_train2020.json` / `instances_attributes_val2020.json` files.

2.2 Preprocessing pipeline and design rationale

The pipeline (`fashionpedia_preprocessing.py`) is deliberately *not* generic — every step encodes a decision grounded in the dataset’s specific pathologies:

1. **Drop zero-attribute annotations** (38.1% of train). Fashionpedia annotators frequently skipped labeling attributes for an instance rather than confirming “no attributes present.”

Metric	Train	Val
Total annotations	333,401	8,781
Total images	45,623	1,158
Annotations with ≥ 1 attribute	206,410 (61.9%)	5,259 (59.9%)
Mean attributes / annotation (raw)	2.28	2.36
Total categories	46	46
Total attributes	294	294

Table 1: Raw Fashionpedia statistics before preprocessing.

Treating skipped instances as confirmed all-negative rows would inject label noise into the BCE loss, so they are removed rather than kept as negatives.

2. **Exclude the *nickname*, *animal*, and *leather* supercategories.** *Nickname* attributes (e.g. “jeans”, “blazer”) are really garment-type identifiers that overlap with the category field and dominate the long tail; *animal* and *leather* have too few positive samples (≤ 210) to be learnable under BCE.
3. **Rarity filter:** attributes with fewer than 300 training samples are dropped as unlearnable given the class-imbalance-aware loss still needs enough positives per batch to produce a gradient signal.
4. **Drop small/degenerate bounding boxes** (area $< 1024 \text{ px}^2$ or side $< 16 \text{ px}$) and duplicate crops.
5. **Train at the annotation (crop) level, not the full image**, since attributes describe individual garment parts, not the photo as a whole.
6. **Compute per-class pos_weight** for BCEWithLogitsLoss from the cleaned label matrix, and build a category $\leftrightarrow$ attribute-supercategory affinity table for optional soft/hard logit masking in future experiments.

Figure 2 diagrams the pipeline; Table 2 quantifies the impact of each cleaning step.

Step	Split	Before	After	Removed	% removed
drop_zero_attr	train	333,401	206,410	126,991	38.1%
drop_small_bbox	train	206,410	190,292	16,118	7.8%
detect_duplicates	train	190,292	190,254	38	0.0%
drop_zero_attr	val	8,781	5,259	3,522	40.1%
drop_small_bbox	val	5,259	4,777	482	9.2%
detect_duplicates	val	4,777	4,776	1	0.0%

Table 2: Impact of each cleaning step on annotation counts (`preprocessing_report.md`).

2.3 Label space and class imbalance

Of the original 294 attributes, excluding *animal*, *leather*, and *nickname* leaves 131; applying the ≥ 300 -sample rarity filter leaves **101 retained attributes** across 8 supercategories. Figure 3 shows the long-tailed frequency distribution and which attributes were retained vs. dropped; Figure 4 shows the final label space broken down by supercategory.

Even after filtering, the retained label space is heavily imbalanced: the BCE positive weight (negative/positive ratio) ranges from 1.38 to 515.5 across the 101 classes (median 92.5, mean 135.8). Figure 5 shows the distribution and the most imbalanced attributes; *paisley*, *queen anne* (neck), and *lace up* are the hardest (rarest) positive classes.

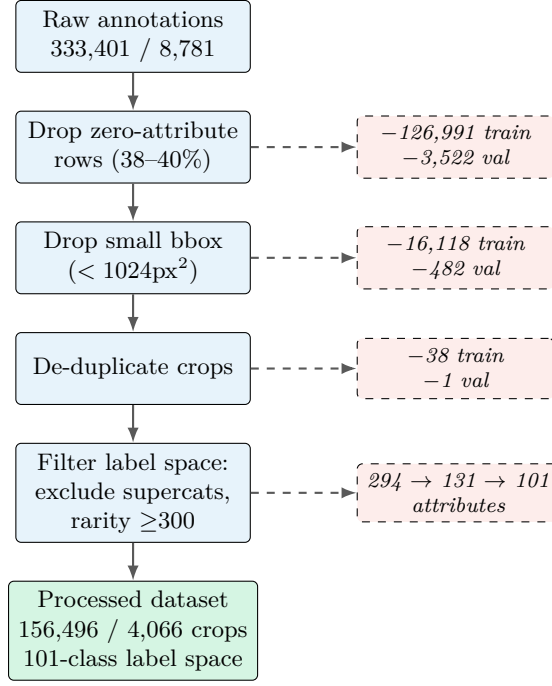


Figure 2: Preprocessing pipeline: sequential cleaning steps and their effect on annotation count and label-space size.

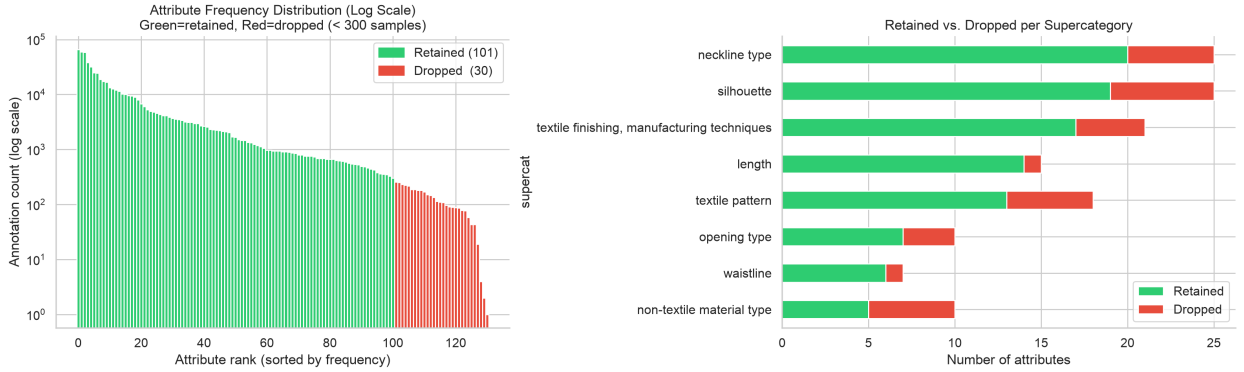


Figure 3: Left: attribute frequency distribution (log scale), green = retained (101), red = dropped for rarity (30). Right: retained vs. dropped attribute counts per supercategory.

2.4 Processed dataset and image/category distribution

The final processed dataset comprises **156,496 training crops** (45,589 unique images) and **4,066 validation crops** (1,158 unique images), each a garment-part bounding-box crop paired with a 101-dimensional multi-hot label vector. Figure 6 shows that *sleeve* dominates the category distribution by a wide margin, followed by *neckline* and *dress*. Figure 7 shows that despite a mean of 3.88 attributes per annotation, the *median* is only 1 — the distribution is bimodal, with a large single-attribute mode and a secondary mode around 7–9 attributes (multi-attribute garment parts like full dresses).

Quality-assurance checks reported zero all-zero label vectors, zero duplicate crops, zero train/val image leakage, and zero missing image files in either split. A category↔attribute-supercategory affinity table (Figure 8) was also computed for possible use as a soft prior or hard logit mask; in the exported artifact all cells evaluate to 0.00, which reflects a normalization convention in the affinity computation rather than genuine independence between categories and attributes, and is flagged here as a known rough edge in the preprocessing pipeline rather than a modeling result.

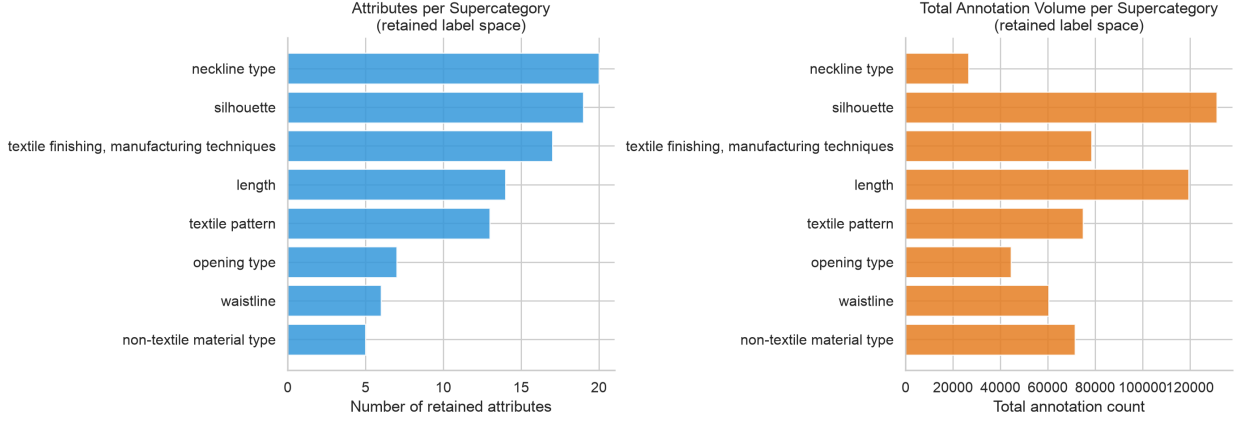


Figure 4: Final 101-attribute label space: number of attributes (left) and total annotation volume (right) per supercategory. *Neckline type* contributes the most distinct attributes (20); *silhouette* and *length* contribute the most annotation volume.

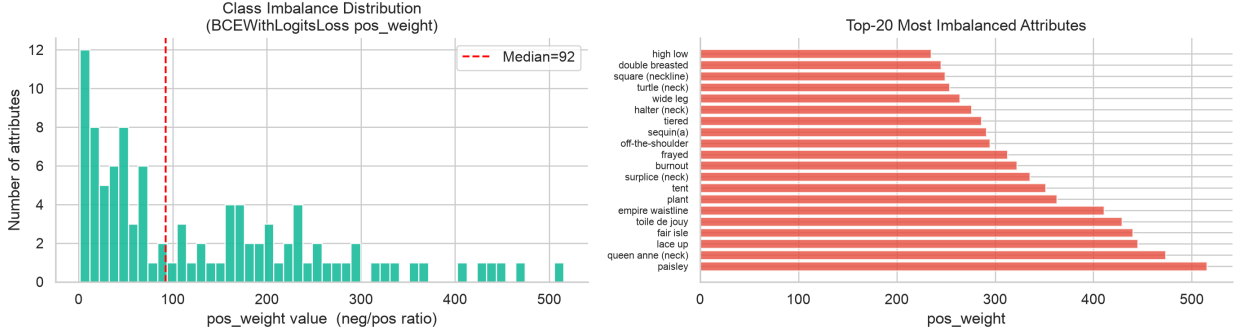


Figure 5: Left: histogram of per-class `pos_weight` (median = 92, dashed line). Right: the 20 most imbalanced retained attributes.

This affinity mask was *not* applied during training (`apply_category_attr_masking` = `False`).

Metric	Train	Val
Annotations (crops)	156,496	4,066
Unique images	45,589	1,158
Label space size	101	101
Mean attrs / annotation	3.88	4.07
Median attrs / annotation	1	1
Annotations with a negation attribute	70,613 (45.1%)	1,900 (46.7%)

Table 3: Final processed dataset statistics.

3 Model Architecture

3.1 Backbone and head

The classifier is a **ConvNeXt-Tiny** (ImageNet-1K-pretrained, ~ 28 M parameters) with its original 1000-way ImageNet head removed, followed by a custom 2-layer MLP head for 101-way multi-label prediction. ConvNeXt-Tiny’s `features` module is a sequential stack of 8 blocks: a stem, four stage blocks, and three downsampling layers interleaved between them (indices 0–7, with index 7 = the deepest / most task-specific stage). The output after adaptive average pooling is a 768-dimensional feature vector.

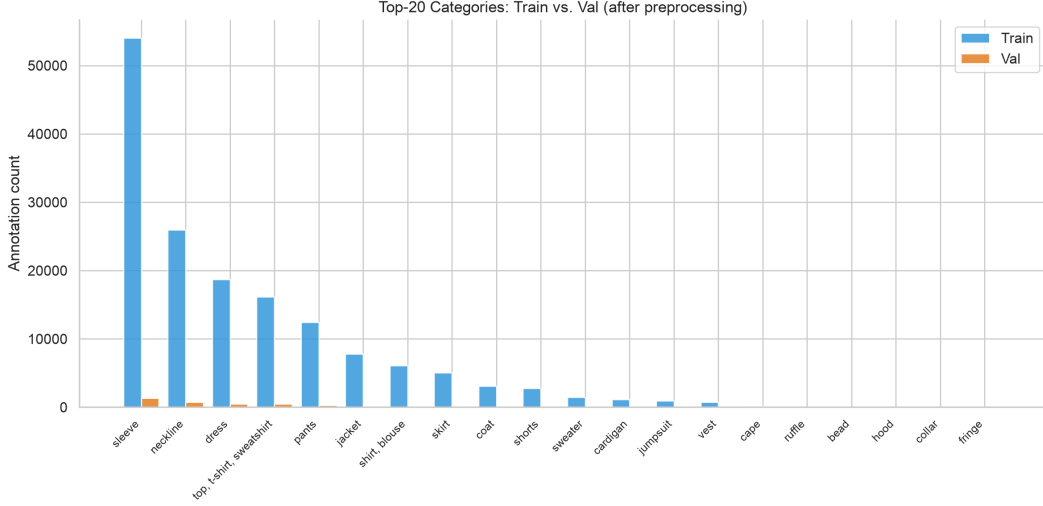


Figure 6: Top-20 garment-part categories after preprocessing, train vs. val.

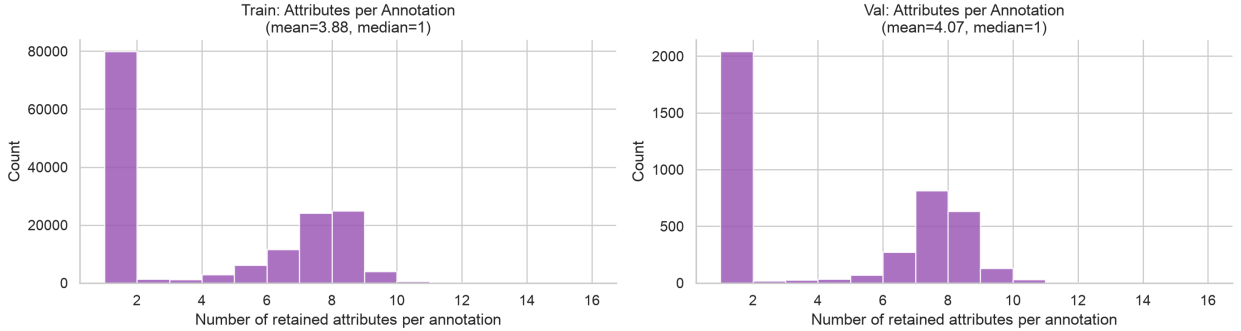


Figure 7: Distribution of number of retained attributes per annotation, train (left) and val (right).

Head component	Shape / operation
LayerNorm	(768) — stabilizes pooled features
Linear	768 → 512
GELU	activation (matches ConvNeXt’s own nonlinearity)
Dropout	$p = 0.3$
Linear	512 → 101 (raw logits, no sigmoid)

The head is a 2-layer MLP rather than a single linear layer because a nonlinear projection lets the model re-compose ImageNet-pretrained visual features into fashion-domain attribute combinations, and dropout regularizes the (randomly initialized, higher-LR) head independently of the backbone’s weight decay.

3.2 Partial fine-tuning and the ablation axis

Training uses a single knob, `unfreeze_from_block`, to control how much of the backbone is trainable: everything in `features[0:unfreeze_from_block]` is frozen (including its LayerNorm parameters — correct for ConvNeXt, which uses LayerNorm rather than BatchNorm), and `features[unfreeze_from_block:8]` plus the head remain trainable. Three configurations were prepared as an ablation study (Table 4); only **Ablation B** was actually trained to a meaningful number of epochs in this repository (see Section 5).

Figure 9 illustrates the backbone/head architecture and which blocks are frozen vs. trainable under the Ablation-B configuration actually used for the reported run.

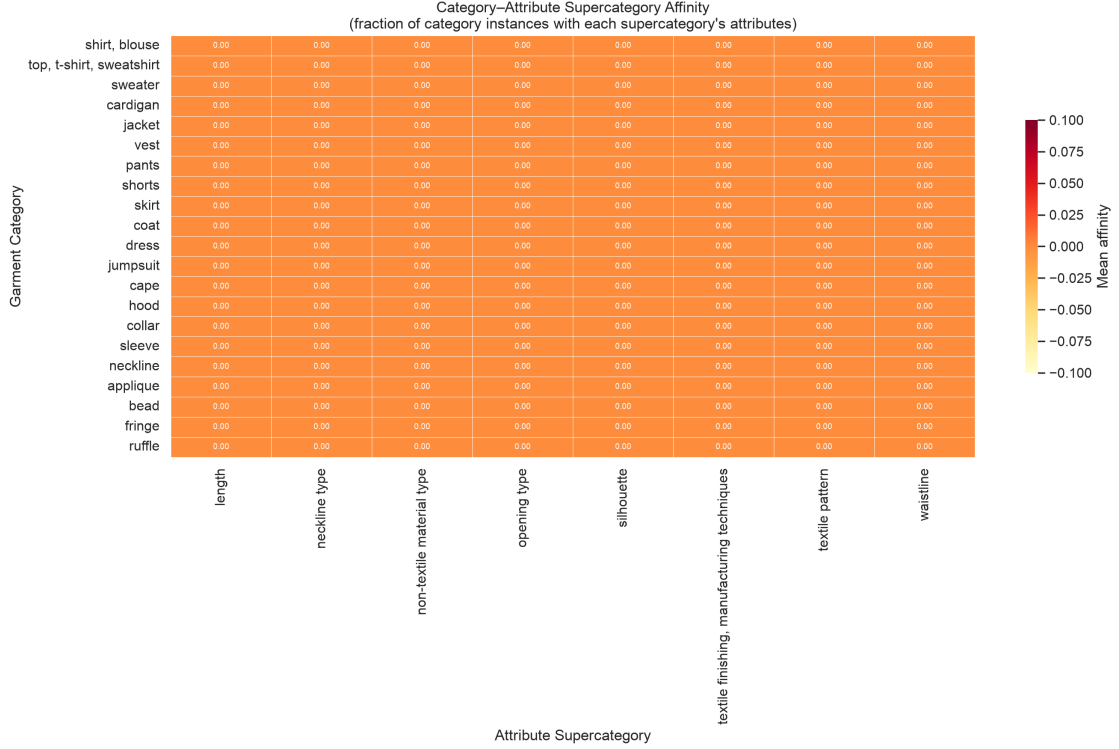


Figure 8: Category×attribute-supercategory affinity heatmap, as exported by the preprocessing pipeline. All cells read 0.00 in this artifact — see caveat above.

Config	unfreeze_from	lr_backbone	Epochs	Warmup	Role
A — head only	8	0 (frozen)	20	3	Lower-bound baseline
B — stage-4 + head	6	1×10^{-4}	30	3	Default / recommended
C — stages 3–4 + head	4	5×10^{-5}	30	5	More capacity, used if B

Table 4: Ablation configurations over how much of the ConvNeXt-Tiny backbone is unfrozen.

3.3 Loss function

Multi-label prediction uses `BCEWithLogitsLoss` with a per-class `pos_weight` tensor computed as $\text{pos_weight}_i = n_{\text{neg},i}/n_{\text{pos},i}$ from the training split, capped at 50 (`pos_weight_cap`) to avoid numerically unstable gradients on the rarest classes. Without this weighting, gradient descent on BCE is dominated by the easy majority-negative signal and the model collapses to near-always-predicting-zero for rare attributes.

3.4 Optimizer and learning-rate schedule

AdamW is used with **differential learning rates**: $\text{lr}_{\text{head}} = 1 \times 10^{-3}$ for the randomly-initialized head, and $\text{lr}_{\text{backbone}} = 1 \times 10^{-4}$ ($10\times$ smaller) for the unfrozen backbone blocks, on the reasoning that pretrained weights should move slowly while the new head adapts quickly. Weight decay (10^{-4}) is applied only to weight matrices, not to biases or LayerNorm parameters. A 3-epoch linear warmup precedes a cosine-annealing schedule (Figure 12) to prevent the initially-random head from destabilizing pretrained backbone features. Gradient norms are clipped to 1.0.

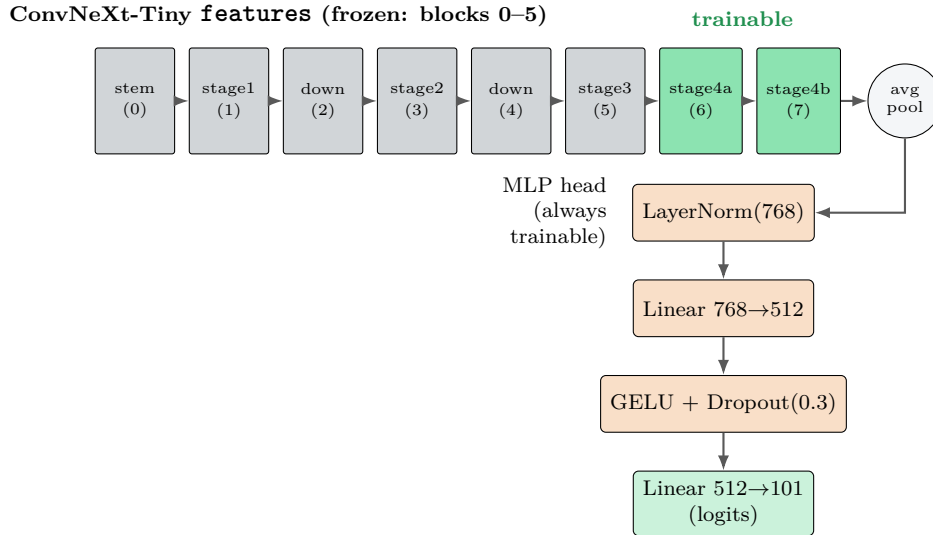


Figure 9: Model architecture under the Ablation-B configuration: ConvNeXt-Tiny blocks 0–5 frozen (grey), blocks 6–7 (“stage 4”) unfrozen (green), followed by the always-trainable MLP head.

4 Training Methodology

4.1 Data pipeline and augmentation

Each training sample is a bounding-box crop resized to 224×224 (ConvNeXt-Tiny’s native input size), normalized with ImageNet statistics. Training augmentation is deliberately curated for fashion imagery:

Used	Rationale
RandomResizedCrop (scale 0.7–1.0)	simulates zoom/framing variation
RandomHorizontalFlip ($p=0.5$)	garments are left-right symmetric
RandomRotation ($\pm 10^\circ$)	small rotational invariance
ColorJitter (mild, hue ≤ 0.05)	lighting variation without corrupting pattern/color attributes
Deliberately avoided	Rationale
Vertical flip	upside-down garments are semantically meaningless
Aggressive color distortion	would destroy texture/pattern attributes (stripe, floral, check)
Cutout / random erasing	can hide the exact region that carries the attribute signal

Validation uses a deterministic resize-then-center-crop transform with no augmentation.

4.2 Metrics

Each epoch reports four validation metrics computed on sigmoid outputs: **mAP** (macro-averaged, threshold-free average precision — the primary model-selection metric), and **macro F1** / **Precision** / **Recall** at a fixed decision threshold (0.5 by default). A final threshold sweep (0.3/0.4/0.5/0.6) is intended to characterize the precision–recall trade-off of the best checkpoint.

4.3 Runs actually executed

Three run directories exist under `fashionpedia_runs/`:

- `run_20260618_040337` and `run_20260618_040727` — 2-epoch *debug* sanity runs (`debug_batches=2`), used to smoke-test the training loop, not meaningful models.

- **ablation_B_stage4_head** — the substantive run, using the Ablation-B configuration (Table 4): batch size 64, MPS device, 30 planned epochs.

Ablations A and C were configured (`config_ablation_A_head_only.json`, `config_ablation_C_stages34_head.json`) but **no corresponding run directories exist** — they were not executed in this repository, so no A/B/C comparison is currently available.

5 Results

5.1 Training run status

The Ablation-B run logged only **7 of the planned 30 epochs** (training log has 7 rows; only one periodic checkpoint, `epoch_005.pt`, plus `best.pt/last.pt`, both from epoch 7). At roughly 24–26 minutes/epoch on MPS, this represents about 2.5 hours of training before the run stopped — there is no evidence of an early-stopping trigger in the code (none is implemented), so the run was most likely interrupted manually or by the environment rather than converging. **Because `val_mAP` was still rising at epoch 7**, `best.pt` and `last.pt` are identical, and the results below should be read as an *early snapshot*, not a converged model. No final threshold sweep exists for this run (`threshold_sweep.csv` is only present for the 2-epoch debug run and is not representative).

Epoch	Train loss	Val loss	Val mAP	Val F1 / P / R
1	0.432	0.461	0.319	0.258 / 0.179 / 0.747
3	0.338	0.451	0.344	0.278 / 0.193 / 0.758
5	0.309	0.451	0.355	0.282 / 0.194 / 0.775
7 (best)	0.278	0.471	0.362	0.291 / 0.202 / 0.746

Table 5: Selected epochs from the Ablation-B training log (`fashionpedia_runs/ablation_B_stage4_head/training_log.csv`).

5.2 Loss and metric curves



Figure 10: Train vs. validation BCE loss over the 7 completed epochs. Validation loss stops improving after epoch 2 and drifts upward from epoch 5 onward, while training loss keeps falling — an early overfitting signal even though the run is short.

5.3 Discussion

Three things stand out from this early snapshot:

1. **Precision/Recall are heavily skewed by `pos_weight`.** At the default 0.5 threshold, macro Recall (≈ 0.75) is roughly $3.7\times$ macro Precision (≈ 0.20). Capping `pos_weight` at 50 (median

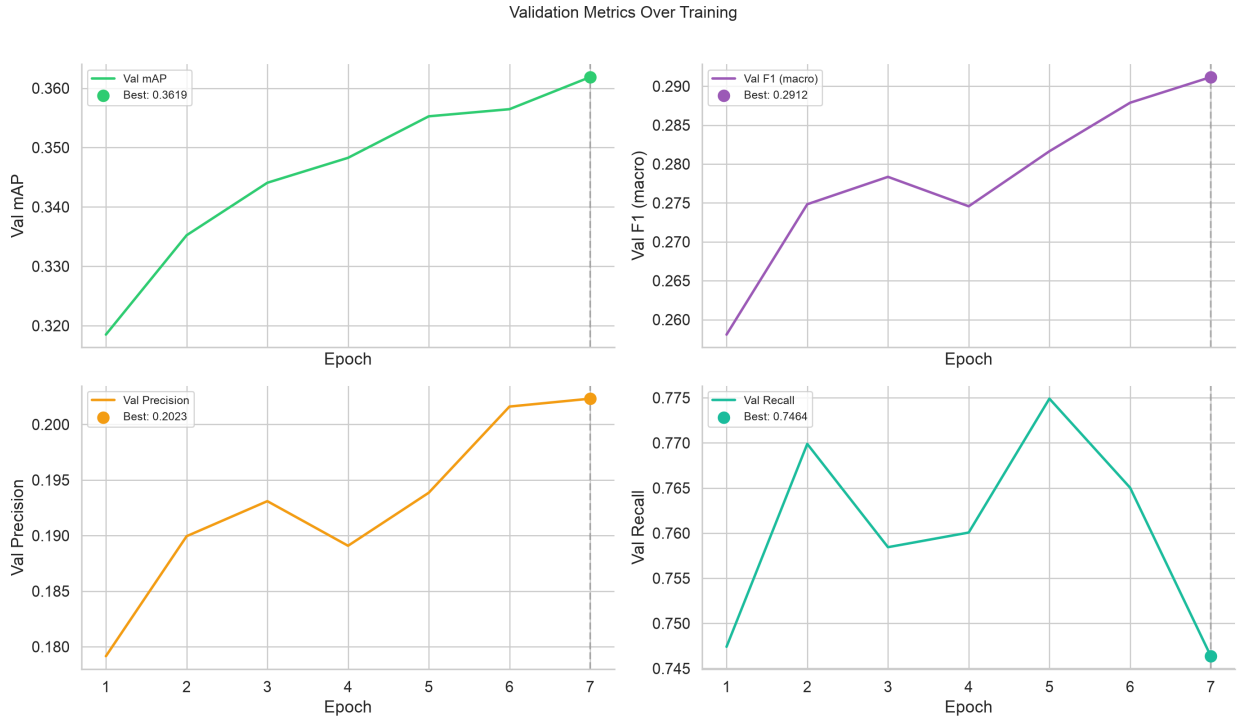


Figure 11: Validation mAP, macro-F1, macro-Precision, and macro-Recall across the 7 completed epochs. mAP and F1 climb monotonically; Precision is low and rising slowly; Recall is high (0.75–0.78) and roughly flat.

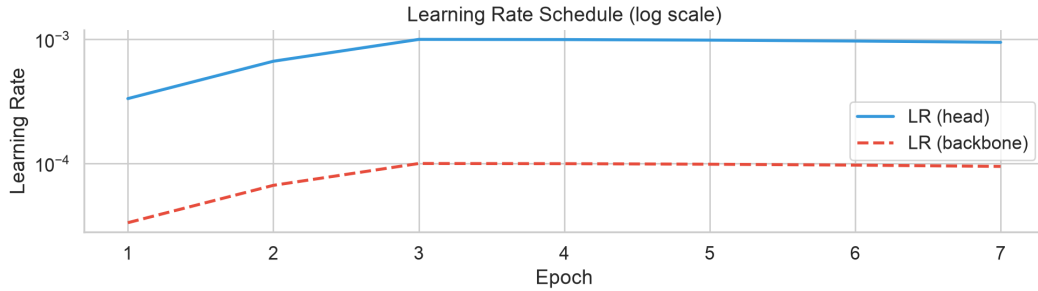


Figure 12: Realized learning-rate schedule for the completed epochs: 3-epoch linear warmup to $\text{lr}_{\text{head}}=10^{-3}$ / $\text{lr}_{\text{backbone}}=10^{-4}$, followed by the start of cosine decay.

class imbalance is 92.5, so more than half of classes are still capped) trades many false positives for fewer false negatives on rare attributes — expected behavior, but it means the model as checkpointed is tuned toward over-prediction, and a threshold sweep well above 0.5 (or per-class thresholds) is needed before this checkpoint is production-ready.

2. **Validation loss already shows an overfitting signal by epoch 5**, even though ranking-based mAP keeps improving. This is a known BCE-with-heavy-pos_weight pattern: the loss is dominated by confident wrong predictions on rare positive classes, which can rise even as the model's overall ranking of positives vs. negatives (mAP) keeps getting better. It is a reason to prefer mAP (the metric actually used for checkpoint selection) over raw val loss when monitoring this run.
3. **No epoch count is long enough to compare Ablation A/B/C**. Since A and C were never run, the claim in the config comments that B is the "good balance" configuration is a design hypothesis, not yet an empirically validated result in this repository.

6 Inference Serving Architecture

The trained checkpoint is served behind a FastAPI application (`main.py`, `app/models.py`) as a three-stage synchronous-per-request pipeline, with all models loaded once at process startup (`PipelineManager.load()`) and reused across requests.

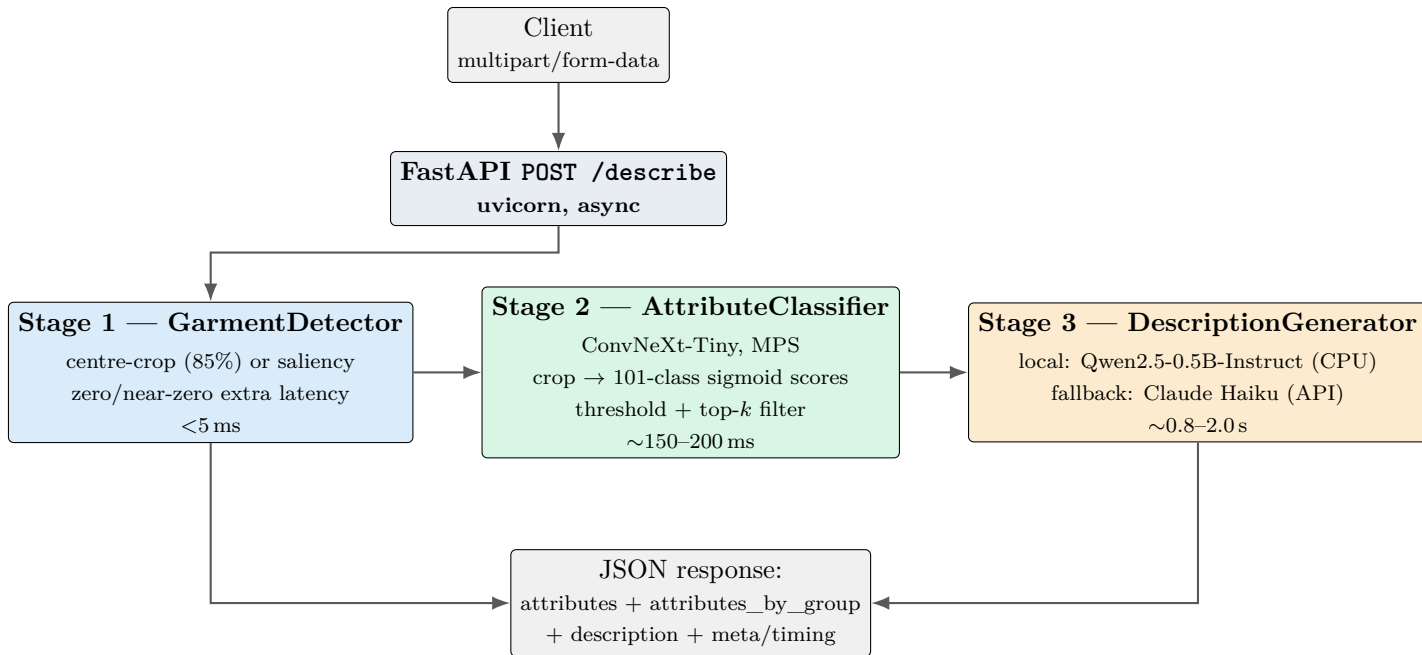


Figure 13: Three-stage inference pipeline behind the `/describe` endpoint, orchestrated by `PipelineManager`.

6.1 Stage details

Stage 1 — Garment detection. The default strategy is a plain **center crop** (85% of the frame) rather than an object detector: e-commerce product photos are composited with the garment already filling most of the frame, so a heavy detector (YOLO/Faster-RCNN) would add 200–500 ms for negligible accuracy gain. A **saliency** strategy (classifier-derived activation map) is available in configuration for non-standard, lifestyle-style photos.

Stage 2 — Attribute classification. The architecture is rebuilt explicitly at load time (rather than unpickling the training class) so the checkpoint stays forward-compatible with code changes; weights are loaded with `weights_only=True` for safety. Predictions above `attribute_threshold` (default **0.45**) are kept, sorted by confidence, and capped to `top_k_attributes` (default **20**); if nothing clears the threshold, the pipeline automatically retries once at threshold 0.3.

Stage 3 — Description generation. Detected attributes are grouped by supercategory and formatted into a structured prompt (e.g. “*Length: midi length (0.87) ... Pattern: floral (0.81) ...*”), then passed to an instruction-tuned LLM with a fixed system prompt asking for a 2–3 sentence product description that does not invent unlisted attributes. Two backends are supported and swappable via `.env` with no code change:

Backend	Model	Notes
local (default)	Qwen2.5-0.5B-Instruct	CPU, Apache-2.0, ~1 GB RAM, ~1.2–2.0 s
claude	Claude Haiku (Anthropic API)	network call, faster end-to-end (~0.8–1.2 s)

The local LLM deliberately runs on **CPU**, not **MPS** — ConvNeXt already occupies the MPS device for Stage 2, and HuggingFace `generate()` on MPS had known KV-cache issues in some

PyTorch versions at the time this was built.

6.2 Latency budget

Stage	Time (M5 MacBook Pro)
Image decode + crop	~5 ms
ConvNeXt-Tiny (MPS)	~150–200 ms
Qwen2.5-0.5B (CPU)	~1.2–2.0 s
Total (local LLM)	~1.5–2.2 s
Total (Claude Haiku backend)	~0.9–1.2 s (network-dependent)

Table 6: Expected end-to-end latency, from README benchmarking on an M5 MacBook Pro.

6.3 API contract

Endpoint	Behavior
POST /describe	multipart image upload (JPEG/PNG/WEBP, max 15 MB) → attributes + grouped attributes + description + inference metadata (crop box, thresholds, model IDs, per-stage timing)
GET /health	liveness/readiness probe reporting model-loaded status, backend, and device

7 Key Engineering Decisions — Summary

- **Crop-level training, not image-level**, because Fashionpedia attributes are properties of garment parts, not whole photos.
- **Drop rather than negate** zero-attribute annotations, to avoid injecting false negatives from annotator skips.
- **Exclude *nickname*** attributes from the visual-attribute head; they are category identifiers, not visual descriptors, and would need a separate head if modeled at all.
- **Partial fine-tuning with differential LR** instead of full fine-tuning or head-only training, to balance adaptation speed against catastrophic forgetting of ImageNet features, with an explicit ablation axis (`unfreeze_from_block`) to make that trade-off tunable.
- **pos_weight-weighted BCE, capped at 50**, to make the 101-class label space (imbalance up to 515×) learnable without exploding gradients on the rarest classes.
- **Center-crop over a learned detector** for garment localization, favoring the empirical structure of e-commerce photography over general-purpose object detection.
- **A small local LLM with an API fallback** for description generation, trading a slightly higher local latency for zero per-request cost and no external dependency, while keeping a faster network-backed option available.

8 Limitations and Future Work

- **The reported model is an early checkpoint (epoch 7/30), not a converged one.** `val_mAP` was still climbing when logging stopped; a full 30-epoch (or longer, with LR re-tuned) run is needed before drawing firm conclusions about Ablation-B’s ceiling.
- **Ablations A and C were never executed**, so the claim that "stage-4 + head" is the right amount of fine-tuning is currently a design hypothesis rather than an empirical result.

- **No final threshold sweep exists for the real model.** Given the precision/recall imbalance already visible by epoch 7, retuning `attribute_threshold` (currently 0.45 in serving, vs. 0.5 in the training-time metric) or moving to per-class thresholds is a likely next step.
- **The category↔attribute affinity table is not being used**, and the exported heatmap artifact currently reads all zeros — worth revisiting if category-conditioned masking is pursued.
- **The center-crop detector is a strong assumption** about photo composition; it will underperform on lifestyle/editorial photography where the garment does not fill the frame, which is what the (currently stubbed-out) saliency strategy is meant to address.
- **No offline evaluation of the generated descriptions exists** (e.g. faithfulness to detected attributes, fluency); Stage 3 is currently validated only by manual inspection of example outputs.

9 Conclusion

This project implements a complete, working pipeline from raw Fashionpedia annotations to a served product-description API: a carefully justified preprocessing pipeline that turns noisy multi-label garment-part annotations into a clean 101-class dataset; a ConvNeXt-Tiny classifier trained with partial fine-tuning, differential learning rates, and imbalance-aware loss; and a three-stage FastAPI inference service that composes the classifier with a local or API-backed LLM for natural-language product copy. The infrastructure — ablation configs, training-curve tooling, structured logging — is in place for a rigorous comparison of fine-tuning strategies; what remains is running the experiments to completion. The single training run captured in this repository (Ablation B, 7 of 30 epochs) already reaches a validation mAP of 0.362 and macro-F1 of 0.291 while still improving, suggesting meaningful headroom is left on the table by the interrupted run.